SQL SERVER — SUJETS PRODUCTION

Grands Sujets en Production

Transactions · Securite · SQL Agent · Monitoring · Plans d'Execution · High Availability

22-26 octobre 2026 • Base : REVISION_DB • SQL Server 2022

CONCEPT CLE MODULES COUVERTS

1. Transactions ACID et TRY/CATCH
2. Verrous et niveaux d'isolation (NOLOCK, SNAPSHOT)
3. Securite — Login vs User, permissions, principe du moindre privilege
4. SQL Server Agent Jobs — automatisation des sauvegardes
5. Monitoring — sessions, blocages, requetes lentes
6. Plans d'execution — Seek vs Scan, Key Lookup, index couvrant
7. High Availability — Log Shipping, Always On AG, RPO/RTO

1. Transactions ACID et TRY/CATCH

Une transaction est un bloc d'operations qui reussit entierement ou echoue entierement. C'est le principe ACID.

Propriete	Signification	Exemple
Atomicity	Tout ou rien	UPDATE + INSERT : si l'un echoue, tout est annule
Consistency	La base reste coherente avant et apres	FK, contraintes UNIQUE respectees apres chaque transaction
Isolation	Les transactions ne se voient pas	Deux sessions ne lisent pas les donnees non validees
Durability	Donnee persistee apres COMMIT	Meme apres redemarrage du serveur

1.1 Pattern production obligatoire — TRY/CATCH

En production, on n'ecrit jamais une transaction sans TRY/CATCH. Sans ca, une erreur laisse la transaction ouverte et bloque les autres sessions.

```
BEGIN TRANSACTION;

BEGIN TRY

    UPDATE dbo.Employes
    SET Salaire = Salaire * 1.10
    WHERE DepartementID = 1;

    INSERT INTO dbo.Employes (Nom, Prenom, Email, Salaire, DepartementID)
    VALUES (N'Dupont', N'Alice', N'alice.dupont@test.com', 3500, 1);

    COMMIT TRANSACTION;
```

```
    PRINT 'Transaction validee.';

END TRY
BEGIN CATCH

    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    PRINT 'Erreur : ' + ERROR_MESSAGE();
    PRINT 'Numero  : ' + CAST(ERROR_NUMBER() AS VARCHAR);

END CATCH;
GO
```

QUESTION D'ENTRETIEN Pourquoi @@TRANCOUNT > 0 avant ROLLBACK ?
Si une erreur survient AVANT le BEGIN TRANSACTION, @@TRANCOUNT = 0.
Un ROLLBACK sans transaction active genere une nouvelle erreur.
On verifie toujours qu'une transaction est bien ouverte avant de l'annuler.

CONCEPT CLE Demonstration du ROLLBACK atomique
Si l'INSERT echoue (email duplique) → le CATCH intercepte → ROLLBACK annule AUSSI l'UPDATE.
Les salaires restent inchanges comme si rien ne s'etait passe.
C'est l'atomicite en action : tout ou rien.

2. Verrous et Niveaux d'Isolation

En production, plusieurs utilisateurs travaillent en meme temps. Les verrous evitent que deux transactions se marchent dessus.

2.1 Les 3 problemes classiques

Probleme	Description	Quand ca arrive
Dirty Read	Lecture d'une donnee non encore validee	Transaction A lit ce que B a modifie mais pas encore commite
Non-Repeatable Read	Meme ligne lue deux fois, resultats differents	B modifie la ligne entre deux lectures de A
Phantom Read	Nouvelle ligne apparait entre deux SELECT	B insere une ligne pendant que A fait deux COUNT(*)

2.2 Niveaux d'isolation

Niveau	Dirty Read	Non-Repeatable	Phantom	Usage
READ UNCOMMITTED	Possible	Possible	Possible	Rapports approximatifs uniquement
READ COMMITTED	Bloque	Possible	Possible	Defaut SQL Server
REPEATABLE READ	Bloque	Bloque	Possible	Rare
SERIALIZABLE	Bloque	Bloque	Bloque	Critique — tres lent
SNAPSHOT	Bloque	Bloque	Bloque	Standard prod moderne

2.3 NOLOCK et SNAPSHOT

```
-- NOLOCK = READ UNCOMMITTED au niveau table
-- Rapide mais peut lire des donnees jamais validees
SELECT * FROM dbo.Employes WITH (NOLOCK);
GO

-- Activer SNAPSHOT ISOLATION sur la base
ALTER DATABASE REVISION_DB
SET ALLOW_SNAPSHOT_ISOLATION ON;
GO

-- Utiliser SNAPSHOT dans une session
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
GO
```

BONNE PRATIQUE DBA SNAPSHOT Isolation — principe
Chaque transaction travaille sur une photo de la base au moment ou elle demarre.
Pas de verrous en lecture — les lecteurs ne bloquent plus les ecrivains.
Solution moderne recommandee pour les applications avec beaucoup de lectures simultanees.

3. Securite et Permissions

3.1 Login vs User — la distinction fondamentale

Concept	Niveau	Role	Commande
LOGIN	SERVEUR	Porte d'entree du serveur SQL Server	CREATE LOGIN ... WITH PASSWORD
USER	BASE	Identite dans une base specifique	CREATE USER ... FOR LOGIN ...

QUESTION D'ENTRETIEN Regle fondamentale
Un LOGIN sans USER dans une base = connexion au serveur possible, mais aucune base accessible.
Un USER sans LOGIN = impossible de se connecter (sauf contained databases).
En production : jamais se connecter avec 'sa'. Toujours un compte applicatif dedie.

3.2 Creer un compte applicatif securise

```
USE master;
GO

-- 1. Login au niveau serveur
CREATE LOGIN app_revision
WITH PASSWORD     = N'Rv@Secure2026!',
     CHECK_POLICY     = ON,   -- Politique mot de passe Windows
     CHECK_EXPIRATION = ON;   -- Expiration activee
GO

USE REVISION_DB;
GO

-- 2. User dans la base
CREATE USER app_revision FOR LOGIN app_revision;
GO

-- 3. Droits minimum (principe du moindre privilege)
GRANT SELECT ON dbo.Employes     TO app_revision;
GRANT SELECT ON dbo.Departements TO app_revision;
GO

-- 4. Tester les restrictions
EXECUTE AS USER = 'app_revision';
GO

SELECT TOP 3 Nom, Prenom FROM dbo.Employes;  -- OK

INSERT INTO dbo.Employes VALUES (...);        -- ERREUR 229 : INSERT refuse

REVERT;
GO
```

3.3 Verifier et gerer les permissions

```
-- Permissions accordees dans la base
SELECT
    dp.name           AS Utilisateur,
```

```
    o.name            AS Objet,
    p.permission_name AS Permission,
    p.state_desc      AS Etat
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
JOIN sys.objects o              ON p.major_id = o.object_id
WHERE dp.name = 'app_revision';
GO

-- Revoquer un droit
REVOKE SELECT ON dbo.Employes FROM app_revision;
GO

-- Voir qui est db_owner (trop de droits en prod)
SELECT dp.name AS Membre, r.name AS Role
FROM sys.database_role_members m
JOIN sys.database_principals dp ON m.member_principal_id = dp.principal_id
JOIN sys.database_principals r  ON m.role_principal_id   = r.principal_id
WHERE r.name = 'db_owner';
GO
```

4. SQL Server Agent Jobs

SQL Agent automatise les taches recurrentes : sauvegardes, maintenance des index, envoi de rapports, nettoyage de logs.

4.1 Creer un job de sauvegarde automatique

```
USE msdb;
GO

-- 1. Creer le job
EXEC sp_add_job
    @job_name = N'REVISION_DB - Sauvegarde FULL Quotidienne';
GO

-- 2. Ajouter une etape avec syntaxe correcte
EXEC sp_add_jobstep
    @job_name  = N'REVISION_DB - Sauvegarde FULL Quotidienne',
    @step_name = N'Backup FULL',
    @command   = N'
DECLARE @Path NVARCHAR(500);
SET @Path = N''C:\SQLServer22\MSSQL16.DBA1\MSSQL\Backup\REVISION_DB_FULL_''
            + CONVERT(VARCHAR, GETDATE(), 112) + ''.bak'';

BACKUP DATABASE REVISION_DB
TO DISK = @Path
WITH COMPRESSION, CHECKSUM, STATS = 10;',
    @on_success_action = 1,
    @on_fail_action    = 2;
GO

-- 3. Planifier tous les jours a 23h00
EXEC sp_add_schedule
    @schedule_name     = N'Quotidien 23h00',
    @freq_type         = 4,       -- Quotidien
    @freq_interval     = 1,
    @active_start_time = 230000;  -- 23:00:00
GO

-- 4. Attacher la planification au job
EXEC sp_attach_schedule
    @job_name      = N'REVISION_DB - Sauvegarde FULL Quotidienne',
    @schedule_name = N'Quotidien 23h00';
GO

-- 5. Enregistrer sur le serveur local
EXEC sp_add_jobserver
    @job_name = N'REVISION_DB - Sauvegarde FULL Quotidienne';
GO
```

4.2 Lancer et verifier un job

```
-- Lancer manuellement
EXEC msdb.dbo.sp_start_job
    @job_name = N'REVISION_DB - Sauvegarde FULL Quotidienne';
GO

-- Verifier l'historique (1=Succes 0=Echec)
SELECT TOP 5
    j.name          AS NomJob,
```

```
    h.step_name     AS Etape,
    h.run_status    AS Statut,
    h.message       AS Message
FROM msdb.dbo.sysjobhistory h
JOIN msdb.dbo.sysjobs j ON h.job_id = j.job_id
WHERE j.name LIKE N'%REVISION_DB%'
ORDER BY h.instance_id DESC;
GO
```

BONNE PRATIQUE DBA Resultat obtenu en lab

Statut 1 = Succes. 658 pages traitees en 0.043s a 119 Mo/s.
Fichier : REVISION_DB_FULL_20261023.bak dans C:\SQLServer22\MSSQL16.DBA1\MSSQL\Backup\
Le job tourne automatiquement chaque nuit a 23h00.

5. Monitoring et Detection des Blocages

En production, ces requetes sont les premieres qu'on lance quand les utilisateurs se plaignent de lenteurs.

5.1 Sessions actives

```
SELECT
    session_id,
    status,
    login_name,
    host_name,
    program_name,
    cpu_time,
    total_elapsed_time / 1000 AS duree_sec,
    last_request_start_time
FROM sys.dm_exec_sessions
WHERE is_user_process = 1
ORDER BY total_elapsed_time DESC;
GO
```

5.2 Requetes en cours — Qui bloque qui ?

```
-- Requetes actives avec blocages
SELECT
    r.session_id,
    r.status,
    r.blocking_session_id,   -- CRITIQUE : 0 = pas bloque
    r.wait_type,
    r.wait_time / 1000 AS attente_sec,
    t.text AS RequeteSQL
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50
ORDER BY r.total_elapsed_time DESC;
GO

-- Afficher uniquement les sessions bloquees
SELECT
    r.session_id              AS SessionBloquee,
    r.blocking_session_id     AS SessionQuiBloque,
    r.wait_type               AS TypeAttente,
    r.wait_time / 1000        AS AttenteSecondes,
    DB_NAME(r.database_id)    AS Base,
    t.text                    AS RequeteBloquee
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id > 0;
GO
```

5.3 Top 10 requetes les plus lentes

```
SELECT TOP 10
    qs.execution_count                                  AS NbExecutions,
    qs.total_elapsed_time / qs.execution_count / 1000 AS DureeMoyenneMs,
    qs.total_logical_reads / qs.execution_count       AS LecturesMoyennes,
    SUBSTRING(qt.text, 1, 200)                        AS RequeteSQL
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
```

```
ORDER BY DureeMoyenneMs DESC;
GO
```

QUESTION D'ENTRETIEN Procedure quand les utilisateurs se plaignent de lenteurs

1. Lancer la requete 'Qui bloque qui' — identifier blocking_session_id > 0.
2. Si blocage : KILL <session_id> en dernier recours apres analyse.
3. Si pas de blocage : lancer Top 10 requetes lentes — identifier la requete fautive.
4. Analyser le plan d'execution de cette requete — chercher un Clustered Index Scan.
5. Creer l'index manquant ou optimiser la requete.

6. Plans d'Execution et Index Couvrants

Le plan d'execution montre comment SQL Server execute une requete. C'est l'outil principal d'optimisation des performances.

6.1 Activer le plan d'execution

```
-- Dans SSMS : Ctrl+M pour activer le plan reel

-- Activer les statistiques IO
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
```

6.2 Les operateurs cles

Operateur	Description	Bon ou Mauvais ?
Index Seek	SQL va directement aux lignes correspondantes	Bon — acces direct
Clustered Index Scan	SQL lit toutes les lignes de A a Z	Mauvais sur grandes tables
Key Lookup	SQL retourne a la table pour colonnes manquantes	A optimiser — cout eleve
Nested Loops	Jointure ligne par ligne	OK sur petits volumes
Hash Match	Jointure par hachage	OK sur grands volumes

6.3 Key Lookup — Probleme et solution

Le Key Lookup apparait quand l'index ne contient pas toutes les colonnes demandees dans le SELECT. SQL doit retourner a la table principale pour les recuperer.

```
-- Requete qui genere un Key Lookup
SELECT Nom, Prenom, Email
FROM dbo.Employes
WHERE Email = N'mamadou.diallo@rv.com';
-- Plan : Index Seek (Email) + Key Lookup (Nom, Prenom) + Nested Loops
GO

-- Solution : Index couvrant avec INCLUDE
DROP INDEX IX_Employes_Email ON dbo.Employes;
GO

CREATE NONCLUSTERED INDEX IX_Employes_Email_Covering
ON dbo.Employes (Email)
INCLUDE (Nom, Prenom);   -- Colonnes supplementaires sans trier
GO

-- Maintenant : Index Seek seul — Key Lookup disparu
SELECT Nom, Prenom, Email
FROM dbo.Employes
WHERE Email = N'mamadou.diallo@rv.com';
```

```
GO
```

	Colonne de cle	INCLUDE
Utilisation	Filtres WHERE, JOIN, ORDER BY	SELECT uniquement
Triee	Oui — dans l'index	Non
Cout	Plus eleve	Plus leger
Exemple	ON dbo.Employes (Email)	INCLUDE (Nom, Prenom)

QUESTION D'ENTRETIEN Question d'entretien — Tu vois un Key Lookup dans un plan. Que fais-tu ?

1. Identifier les colonnes que SQL va chercher dans la table (visibles dans le plan).
2. Les ajouter dans un INCLUDE sur l'index existant.
3. Relancer la requete et verifier que le Key Lookup a disparu.

Resultat obtenu : passage de 3 operateurs a 1 seul Index Seek.

CONCEPT CLE Pourquoi l'optimiseur choisit parfois un Scan meme avec un index ?

Si la requete retourne un grand pourcentage de la table (>20-30%),
SQL juge qu'il est plus rapide de scanner la table entiere
que de naviguer dans l'index puis faire des Key Lookups pour chaque ligne.
L'optimiseur a toujours le dernier mot — il choisit le plan le moins couteux.

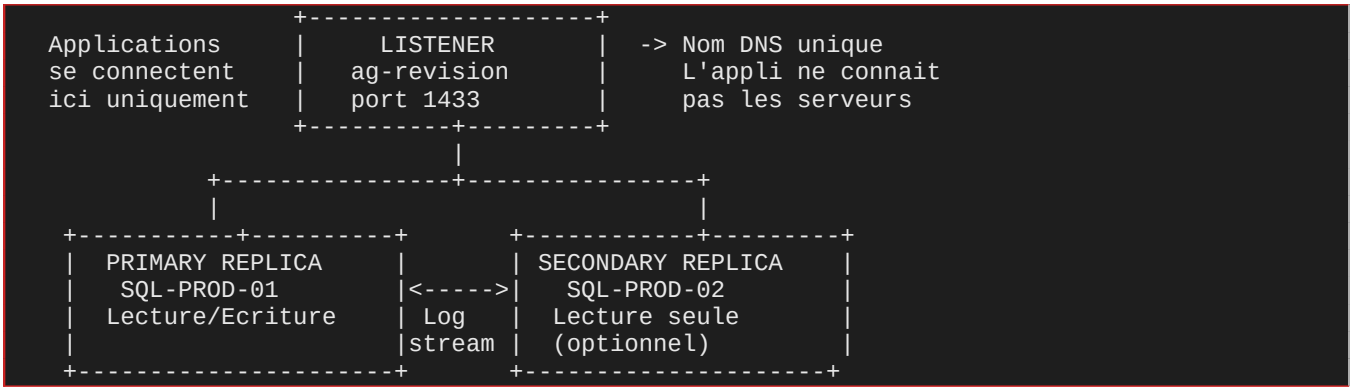
7. High Availability — Always On AG

La haute disponibilite garantit la continuite de service meme en cas de panne du serveur principal.

7.1 Les 3 solutions SQL Server

Solution	Depuis	Statut	Usage
Mirroring	SQL 2005	Deprecie	Ne plus utiliser
Log Shipping	SQL 2000	Toujours valide	Simple, pas de failover auto
Always On AG	SQL 2012	Solution actuelle	Standard en production

7.2 Architecture Always On



7.3 Synchronous vs Asynchronous Commit

Mode	Comportement	RPO	Performance
Synchronous	Transaction validee sur PRIMARY et SECONDARY avant confirmation	0 — zero perte	Plus lent
Asynchronous	Transaction validee sur PRIMARY, secondaire suit apres	Quelques secondes	Plus rapide

BONNE PRATIQUE DBA Regle prod pour les modes de commit
Synchronous : secondaire dans le MEME datacenter — latence faible, RPO = 0.
Asynchronous : secondaire dans un AUTRE datacenter (DR) — latence reseau, performance preservee.
On peut avoir les deux en meme temps : un sync local + un async DR.

7.4 RPO et RTO — Les metriques cles

Metrique	Definition	Always On Synchrone	Sauvegarde seule
RPO — Recovery Point Objective	Donnees perdues maximum	0 seconde	Jusqu'a 24h
RTO — Recovery Time Objective	Temps de retour en ligne	10-30 secondes	Des heures

7.5 Verifier l'etat des replicas

```
-- Vue des replicas Always On (necessite AG configure)
SELECT
    ag.name                          AS GroupeDisponibilite,
    ar.replica_server_name           AS Serveur,
    ars.role_desc                    AS Role,   -- PRIMARY / SECONDARY
    ars.synchronization_health_desc  AS Sante,
    ars.connected_state_desc         AS Connexion
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id;
GO
```

QUESTION D'ENTRETIEN Questions d'entretien Always On

Q : Difference Always On vs Mirroring ?
R : Mirroring = deprecie, 1 seul secondaire, pas de lecture sur secondaire.
Always On = 8 secondaires max, lecture sur secondaires, failover auto, listener DNS.

Q : RPO = 0, comment on fait ?
R : Always On Synchronous Commit. Transaction confirmee apres validation sur les 2 replicas.

Q : L'application doit connaitre les noms des serveurs ?
R : Non. Elle se connecte au Listener. En cas de failover, le listener bascule automatiquement.

8. Recapitulatif — Questions d'Entretien

Question	Reponse courte
Pourquoi TRY/CATCH sur chaque transaction ?	Sans ca, une erreur laisse la transaction ouverte et bloque les autres sessions indefiniment.
Pourquoi @@TRANCOUNT > 0 avant ROLLBACK ?	Si l'erreur arrive avant BEGIN TRANSACTION, @@TRANCOUNT = 0. ROLLBACK sans transaction = nouvelle erreur.
Difference NOLOCK vs SNAPSHOT ?	NOLOCK = lecture sans verrou, peut lire des donnees non validees. SNAPSHOT = photo coherente de la base, pas de lecture sale.
Login vs User SQL Server ?	Login = acces au serveur. User = acces a une base specifique. Un login sans user = impossible d'acceder a une base.
Principe du moindre privilege ?	Un compte applicatif ne doit avoir que les droits strictement necessaires. SELECT seul si pas besoin d'ecrire.
Comment automatiser les sauvegardes ?	SQL Server Agent Job avec sp_add_job, sp_add_jobstep, sp_add_schedule, sp_attach_schedule, sp_add_jobserver.
Tu vois un Key Lookup dans un plan — que fais-tu ?	Identifier les colonnes manquantes dans l'index et les ajouter dans un INCLUDE pour creer un index couvrant.
Difference Index Seek vs Scan ?	Seek = acces direct aux lignes correspondantes. Scan = lecture de toutes les lignes. Seek = rapide, Scan = lent sur grandes tables.
Quand l'optimiseur choisit un Scan malgre un index ?	Quand la requete retourne >20-30% des lignes. SQL juge le scan plus rapide que seek + key lookups multiples.
RPO = 0 — comment on fait ?	Always On Availability Groups en mode Synchronous Commit. Transaction validee sur PRIMARY et SECONDARY simultanement.
Difference Always On vs Mirroring ?	Mirroring deprecie depuis 2012. Always On : 8 secondaires, lecture sur secondaires, failover auto, listener DNS.
RTO Always On vs restauration manuelle ?	Always On : 10-30 secondes de failover automatique. Restauration manuelle : des heures selon la taille de la base.